

Artificial Intelligence (CS-203)

Objectives:

- Segment matching
 - ELIZA code
-

Segment matching:

In this week we want to extend the pattern matching module (i.e. `match_segment` function) to handle the segment matching. In segment matching a logical variable can match with more than one symbols (or even with an empty segment). So now we shall call such a logical variable (which will be used in `match_segment`) as 'segment variable'. A segment variable will be denoted by `?*var`. In other words a logical variable which can bound with exactly one symbol has the prefix `?` while a segment variable which can bind with zero or more symbols has the prefix `?*` (note that `*` is additional in segment variable).

So now if we have a following rule pattern;

Pattern: (`?*a my name is ?*name`) It can match will all the following user inputs;

'Hello my name is Asif Ali Khan'

Binding: `{'a': ['Hello'], 'name': ['Asif', 'Ali', 'Khan']}`

'Hi my name is Asif'

Binding: `{'a': ['Hi'], 'name': ['Asif']}`

'My name is Fahad Tariq'

Binding: `{'a': [ ], 'name': ['Fahad']}`

Note that in last call segment variable `'a'` has bound with empty segment.

1) `is_segment` function: to check first element of a list is segment variable or not

```
def is_segment(pattern):
    """Test if pattern begins with a segment variable."""
    return (type(pattern) is list
            and pattern
            and len(pattern[0]) > 2
            and pattern[0][0] == '?'
            and pattern[0][1] == '*'
            and pattern[0][2] in letters
            and ' ' not in pattern[0])
```

```

>>> is_segment("abc")
False
>>> is_segment("?abc")
False
>>> is_segment("?*abc")
False
>>> is_segment(["?*abc"])
True
>>> is_segment(["?abc"])
False
>>> is_segment(["?*a"])
True
>>> is_segment(["?*a", 'hello', '?*del'])
True

```

2) Match_Segment : Segment matching function

```

def match_segment(var, pattern, input, bindings):
    if not pattern:
        return match_variable(var, input, bindings)
    word = pattern[0]
    try:
        pos = input[0:].index(word)
    except ValueError:
        return False
    var_match = match_variable(var, input[:pos], dict(bindings))
    match = match_pattern(pattern, input[pos:], var_match)
    return match

>>> bindings = {}
>>> match_segment('?*a', ['hello', 'eliza'], ['hello', 'eliza'],
bindings)
{'?*a': []}
>>> match_segment('?*a', ['hello', 'eliza'], ['test', 'hello',
'eliza'], bindings)
{'?*a': ['test']}
>>> match_segment('?*a', ['hello', '?b'], ['test', 'hello', 'eliza'],
bindings)
{'?*a': ['test'], 'b': ['eliza']}
>>> match_pattern(['a', '?d', '?e'], ['a', 'b', 'c'])
{'d': ['b'], 'e': ['c']}
>>> match_segment('?*a', ['hello', '?*b'], ['test', 'hello', 'eliza',
'how', 'are', 'you'], bindings)
{'?*a': ['test'], 'b': ['eliza', 'how', 'are', 'you']}

```

3) Match_Pattern : pattern_match function redefined to include Segment matching function

```

def match_pattern(pattern, input, bindings=None):
    if bindings is False:

```

```

        return False
    if pattern == input:
        return bindings
    bindings = bindings or {}
    if is_segment(pattern):
        token = pattern[0] # segment variable is the first token
        var = token[2:] # segment variable is of the form ?*x
        return match_segment(var, pattern[1:], input, bindings)
    elif is_variable(pattern):
        var = pattern[1:]
        return match_variable(var, [input], bindings)
    elif contains_tokens(pattern) and contains_tokens(input):
        return match_pattern(pattern[1:], input[1:],
                               match_pattern(pattern[0], input[0], bindings))
    else:
        return False

def match(p, i):
    return match_pattern(p.split(), i.split())

print(match_pattern(['a', '?x', '?*y', 'e', 'f'], ['a', 'b', 'c', 'd', 'e', 'f']
))

```

P: ['a', '?x', '?*y', 'e', 'f'] I: ['a', 'b', 'c', 'd', 'e', 'f'] B: None

P: 'a' I: 'a' B: {}

P: ['?x', '?*y', 'e', 'f'] I: ['b', 'c', 'd', 'e', 'f'] B: {}

P: '?x' I: 'b' B: {}

P: ['?*y', 'e', 'f'] I: ['c', 'd', 'e', 'f'] B: {'x': ['b']}

P: ['e', 'f'] I: ['e', 'f'] B: {'x': ['b'], 'y': ['c', 'd']}

{'x': ['b'], 'y': ['c', 'd']}

```

print(match ( '?*x studying ?*d from ?*n days',
              'I am studying online courses from last 60 days'))

```

P: ['?*x', 'studying', '?*d', 'from', '?*n', 'days']

I: ['I', 'am', 'studying', 'online', 'courses', 'from', 'last', '60', 'days'] B: None

P: ['studying', '?*d', 'from', '?*n', 'days'] I: ['studying', 'online', 'courses', 'from', 'last', '60', 'days'] B: {'x': ['I', 'am']}

P: ['?*d', 'from', '?*n', 'days'] I: ['online', 'courses', 'from', 'last', '60', 'days'] B: {'x': ['I', 'am']}

P: 'studying' I: 'studying' B: {'x': ['I', 'am']}

P: ['from', '?*n', 'days'] I: ['from', 'last', '60', 'days'] B: {'x': ['I', 'am'], 'd': ['online', 'courses']}

P: ['?*n', 'days'] I: ['last', '60', 'days'] B: {'x': ['I', 'am'], 'd': ['online', 'courses']}

P: 'from' I: 'from' B: {'x': ['I', 'am'], 'd': ['online', 'courses']}

P: ['days'] I: ['days'] B: {'x': ['I', 'am'], 'd': ['online', 'courses'], 'n': ['last', '60']}

{'x': ['I', 'am'], 'd': ['online', 'courses'], 'n': ['last', '60']}

Eliza Complete Code

```

letters = "abcdefghijklmnopqrstuvwxyzABCEDFGHIJKLMNOPQRSTUVWXYZ"
def is_variable(pattern):
    return (type(pattern) is str
            and pattern[0] == '?'
            and len(pattern) > 1
            and pattern[1] != '*'
            and pattern[1] in letters
            and ' ' not in pattern)
#####
def contains_tokens(pattern):
    return type(pattern) is list and len(pattern) > 0
#####
def match_variable(var, replacement, bindings):
    binding = bindings.get(var)
    if not binding:
        bindings.update({var: replacement})
        return bindings
    if replacement == bindings[var]:
        return bindings
    return False
#####
def is_segment(pattern):
    return (type(pattern) is list
            and pattern and len(pattern[0]) > 2
            and pattern[0][0] == '?' and pattern[0][1] == '*'
            and pattern[0][2] in letters and ' ' not in pattern[0])
#####
def match_segment(var, pattern, input, bindings, start=0):
    if not pattern:
        return match_variable(var, input, bindings)
    word = pattern[0]
    try:
        pos = start + input[start:].index(word)
    except ValueError:
        return False
    var_match = match_variable(var, input[:pos], dict(bindings))
    match = match_pattern(pattern, input[pos:], var_match)
    if not match:
        return match_segment(var, pattern, input, bindings, start + 1)
    return match
#####
def match_pattern(pattern, input, bindings=None):
    if bindings is False:
        return False
    if pattern == input:
        return bindings

```

```

bindings = bindings or {}
if is_segment(pattern):
    token = pattern[0]
    var = token[2:]
    return match_segment(var, pattern[1:], input, bindings)
elif is_variable(pattern):
    var = pattern[1:]
    return match_variable(var, [input], bindings)
elif contains_tokens(pattern) and contains_tokens(input):
    return match_pattern(pattern[1:], input[1:], match_pattern(pattern[0],
input[0], bindings))
else:
    return False
#####
rules = [
    ("?*X HELLO ?*Y", [ "How do you do. Please state your problem."]),
    ("?*X COMPUTER ?*Y", [ "Do computers worry you?",
        "What do you think about machines?",
        "Why do you mention computers?",
        "What do you think machines have to do with your problem?",]),
    ("?*X NAME IS ?*Y", [ "I am not interested in names",
        "what do you do ?Y"]]),
    ("?*X NEED ?*Y", ["What if ?X get ?Y", "why ?X need ?Y"]])
default_responses = [ "Very interesting",
    "I am not sure I understand you fully",
    "Please continue" ]
#####
import random
def replace(word, replacements):
    for old, new in replacements:
        if word == old:
            return new
    return word
#####
def switch_viewpoint(words):
    replacements=[('I', 'YOU'), ('YOU', 'I'), ('ME', 'YOU'),
        ('MY', 'YOUR'), ('AM', 'ARE'), ('ARE', 'AM')]
    return [replace(word, replacements) for word in words]
#####
def respond(rules, input, default_responses):
    input = input.split()
    matching_rules = []
    for pattern, responses in rules:
        pattern = pattern.split()
        bindings = match_pattern(pattern, input)
        if bindings:
            matching_rules.append((responses, bindings))
    if matching_rules:
        responses, bindings = random.choice(matching_rules)
        response = random.choice(responses)
    else:
        bindings = {}
        response = default_responses[0]
    for variable, value in bindings.items():
        value = ' '.join(switch_viewpoint(value))
        if value:
            response = response.replace('?' + variable, value)

```

```
        return response
#####
def Eliza(prompt, rules, default_responses):
    while True:
        try:
            userinput = input(prompt).upper()
            if not userinput:
                continue
        except:
            break
        print(respond(rules, userinput, default_responses))
    Eliza('Eliza> ', rules, default_responses)
```